

Module:4 Programming Tools

Evolution of embedded programming tools, Modelling programs, Code optimization, Logic analyzers, Programming environment.

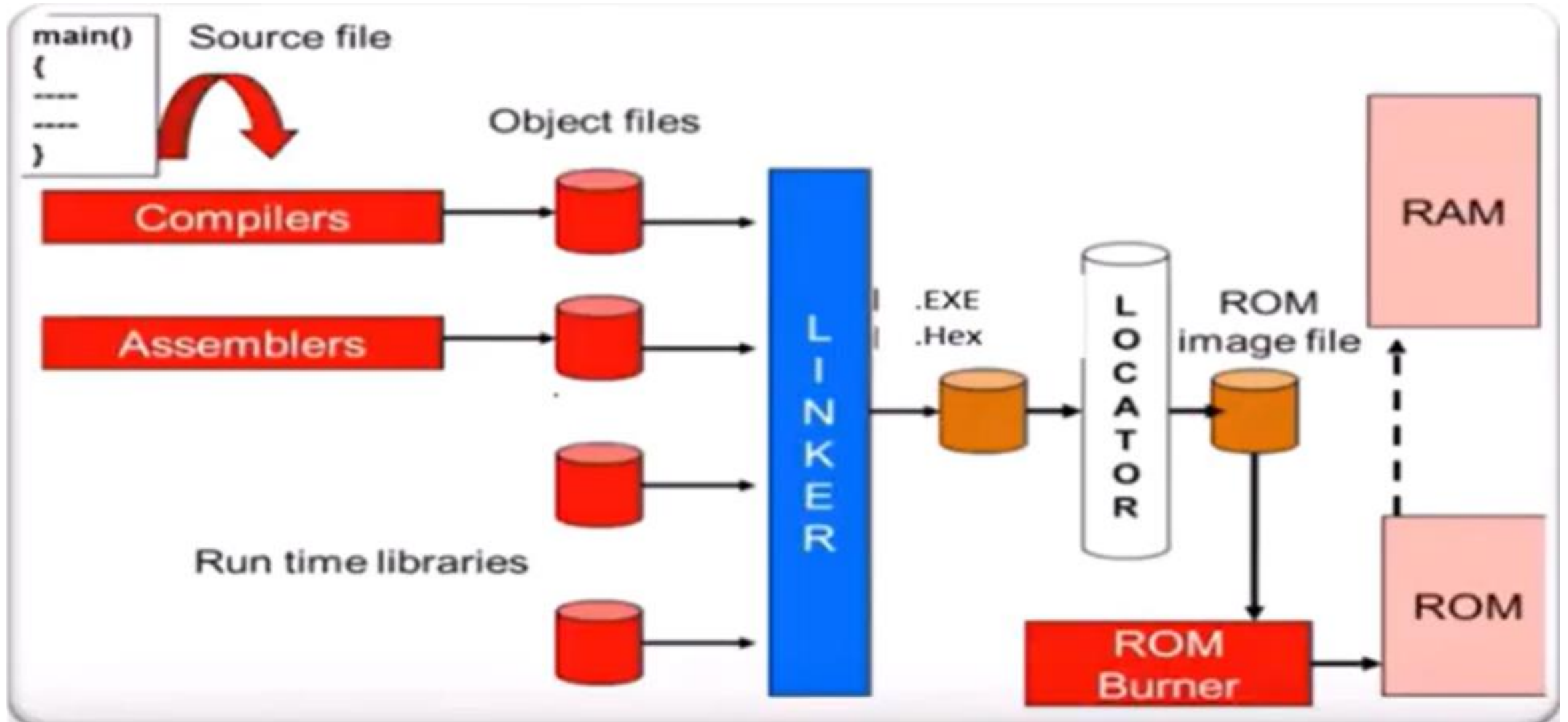
Module:5 Real Time Operating System :

Classification of Real time system, Issues & challenges in RTS, Real time scheduling schemes- EDF-RMS & Hybrid techniques, eCOS, POSIX, Protothreads.

Embedded software development tools

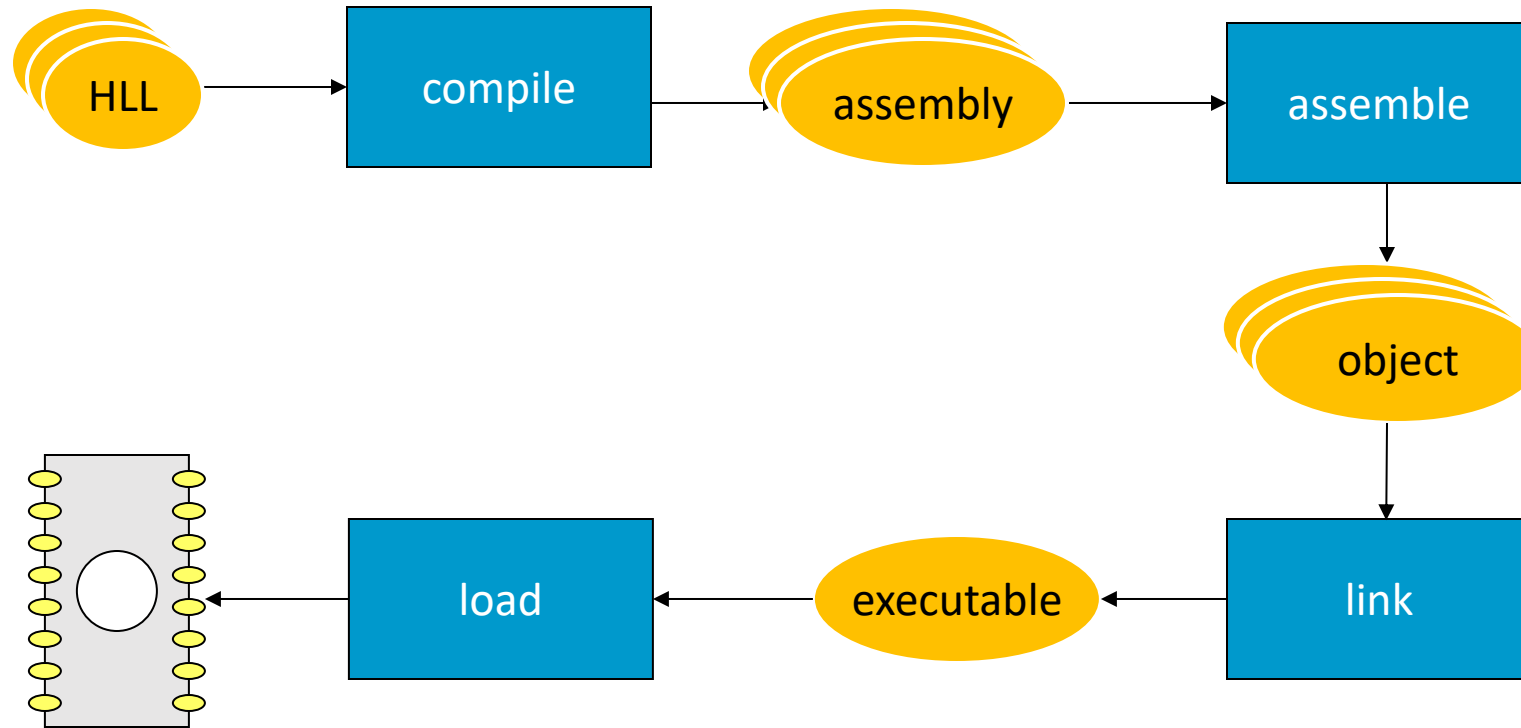
- 1.Editor
- 2.Assembler/Compiler
- 3.Linkers
- 4.Debugger
- 5.Simulator
- 6 Locator
- 7.Downloader
- 8.IDE

Flow of Software Tools

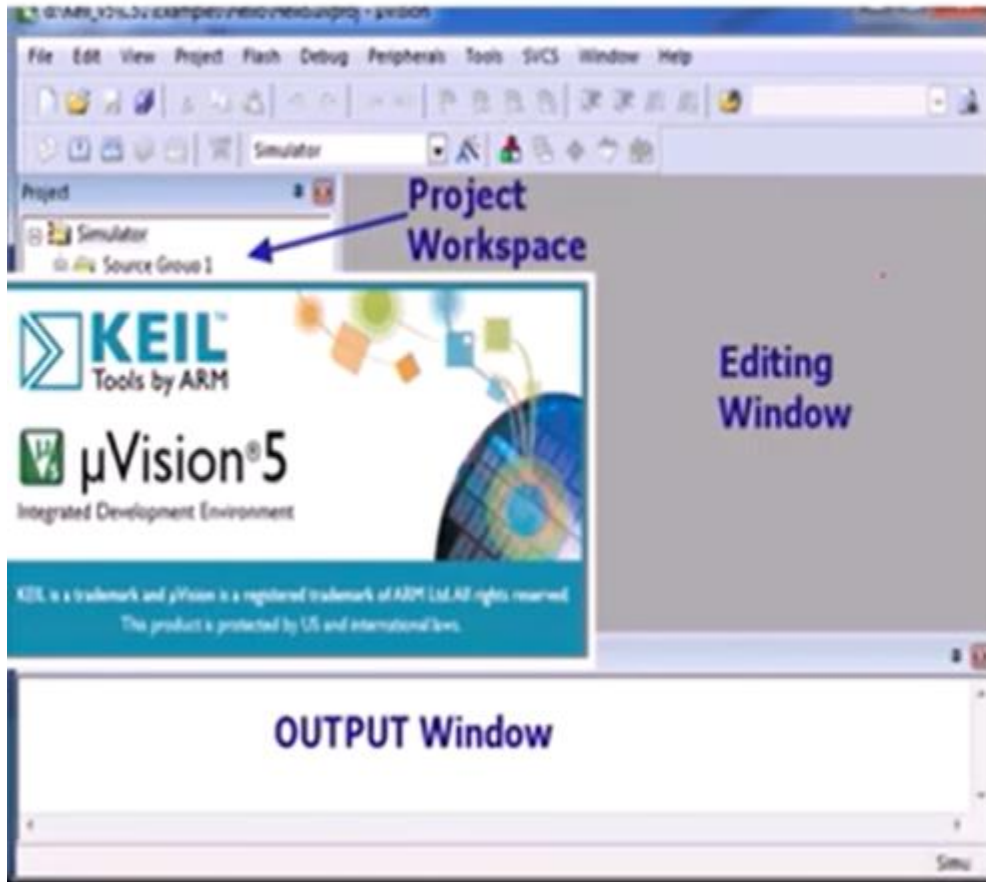


Assembly and linking

- Last steps in compilation:



Integrated Development Environment (IDE)



- Integrated Development Environment Software usually consists of the debugger, compiler, and a code editor.
- There are many Integrated Development Environment is available in the market but your choice is dependent on the type of microcontroller and processor you are using for your development.
- Examples :
 - MPLAB
 - Keil MDK
 - Xcode
 - Eclipse
 - Android Studio
 - Code Blocks
 - Adobe Flash Builder etc.
 - BlueJ

<https://www.circuitgallery.com/how-to-simulate-pic-microcontroller-in-proteus/>

<https://www.tinkercad.com/> <https://www.electronicwings.com/arm7/getting-started-with-arm-lpc2148-using-keil-uvision-ide>

Models of programs

- Source code is not a good representation for programs:
 - clumsy;
 - leaves much information implicit.
- Compilers derive intermediate representations to manipulate and optimize the program.

<https://www.sam-solutions.com/blog/top-ten-embedded-software-development-tools/>

Computational Models

What:

- A conceptual notion for expressing the function of a system
E.g. DFG, FSM, Petri net, Turing machine, etc.

Computational Models & Languages

- Models express the behavior, languages capture models
- Models are conceptual, languages are concrete

What is in a computational model

- A set of objects
- Rules
- Semantics

Computational Models in Embedded Design

- The commonly used computational models in embedded system design are:
 - Data Flow Graph Model
 - Control Data Flow Graph Model
 - State Machine Model
 - Sequential Program Model
 - Concurrent/Communicating Process Model
 - Object-Oriented Model

Embedded applications which are **computational intensive and data driven** are modeled using the DFG model.

Data Flow Graph

Definition

- A directed graph that shows the **data dependencies** between a number of functions

- $G=(V,E)$

Nodes (V): each node having input/output data ports

Arces (E): connections between the output ports and input ports

- Semantics

Fire when input data are ready

Consume data from input ports and produce data to its output ports

There may be many nodes that are ready to fire at a given time

Data driven model where program execution is driven by the data It translate the requirement in the data flow

Data Flow Graph/Diagram (DFG) Model

- **The Data Flow Graph (DFG) model translates the data processing requirements into a data flow graph**
- . • It is a data driven model in which the program execution is determined by data.
- This model emphasises on the data and operations on the data which transforms the input data to output data.
- Embedded applications which are computational intensive and data driven are modelled using the DFG model. • DSP applications are typical examples for it. (operations and speech recognition etc

Data Flow Graph (DFG) is a visual model in which the operation on the data (process) is represented using a block (circle) and data flow is represented using arrows.

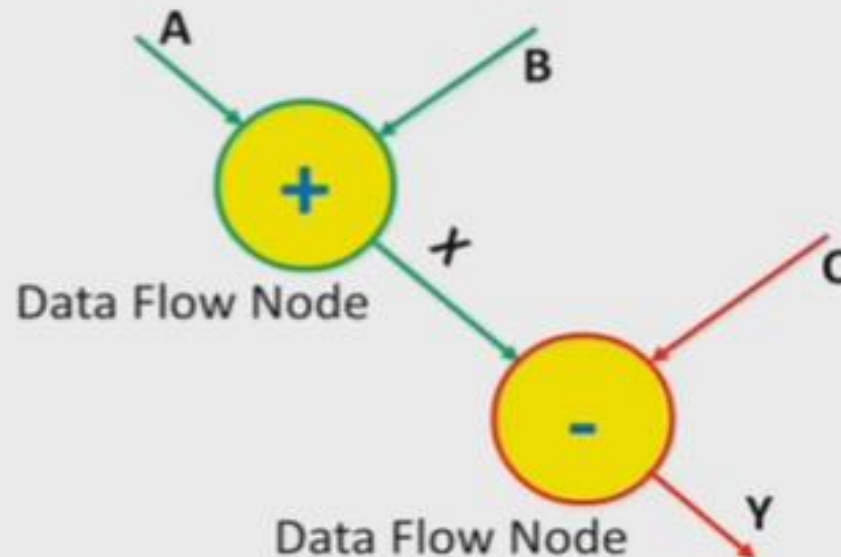
An inward arrow to the process (circle) represents input data and an outward arrow from the process (circle) represents output data in DFG notation.

Suppose one of the functions in our application contains the computational requirement

$$x = a + b \text{ and } y = x - c$$

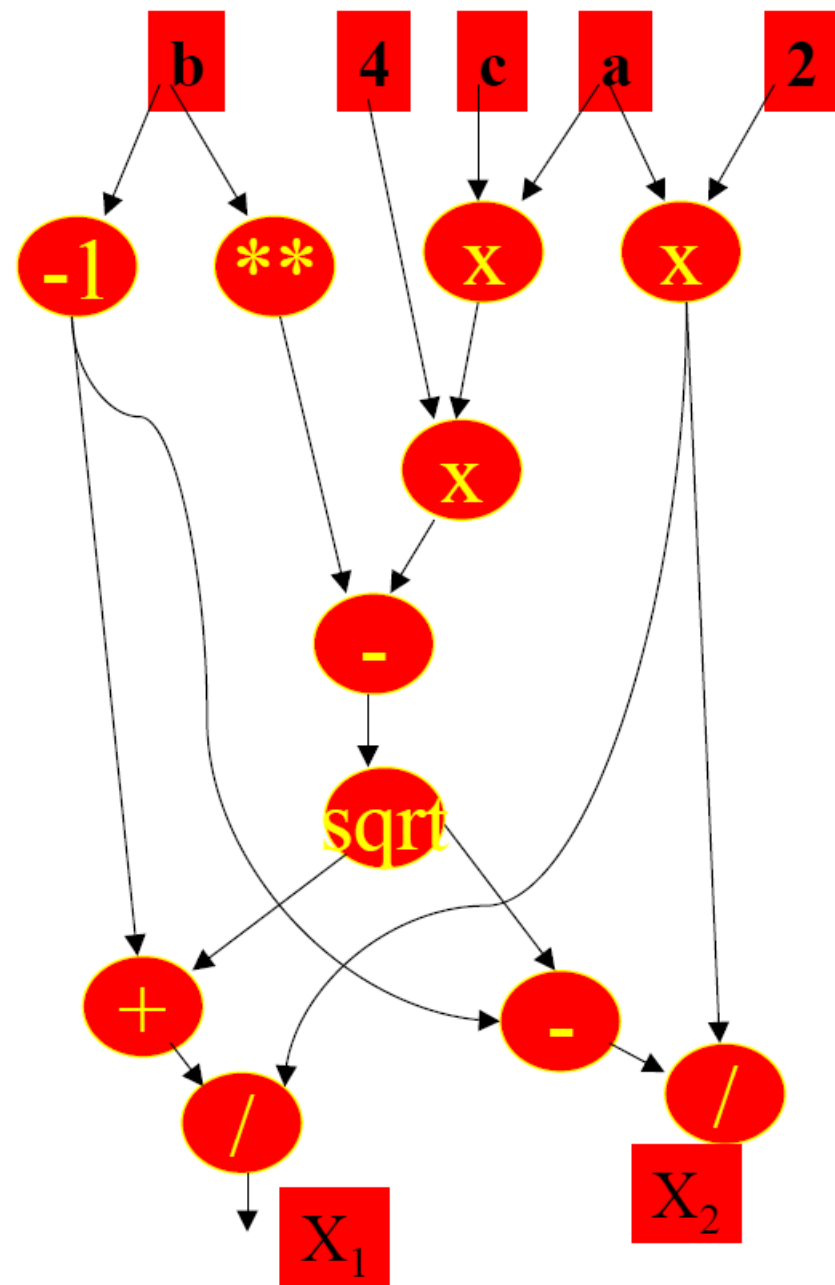
Example of Data Flow Graph Model

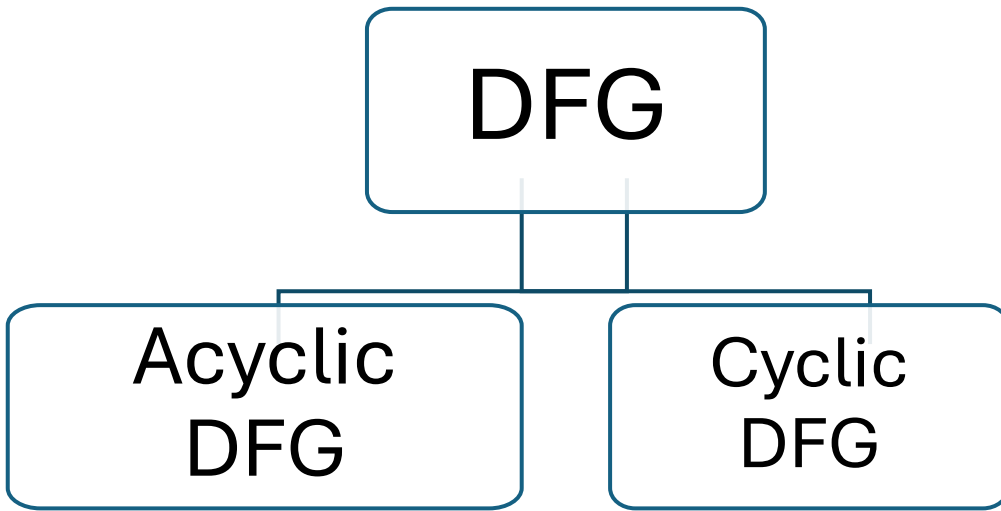
- ☐ Inward arrow to circle ----- INPUT DATA
- ☐ Outward arrow to circle ----- OUTPUT DATA
- ☐ Example $X = A+B$ and $Y = X-C$



$$x_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$$

$$x_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

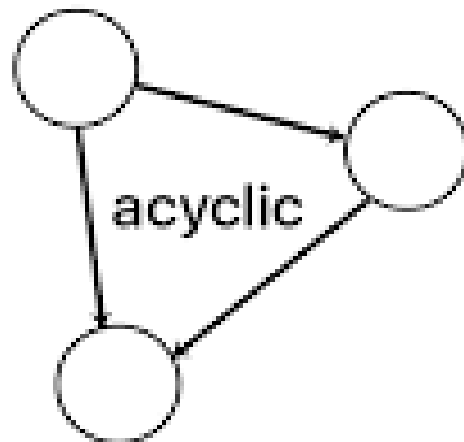
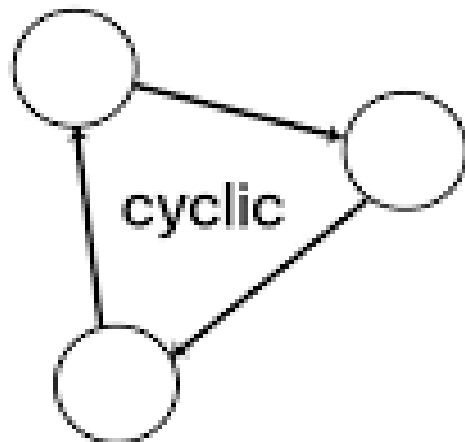




Acyclic DFG (ADFG)

- doesn't contain multiple values for the input variable and multiple output values for a given set of inputs
- there is no cycle, or closed path

Directed Graphs



non-acyclic DFG

Output is fed back to Input

In a DFG model, a data path is the data flow path from input to output.

There are two types of DFG

- 1. A DFG model is said to be **Acyclic DFG (ADFG)** if it doesn't contain multiple values for the input variable and multiple output values for a given set of input(s) **(No multiple values, no feedback, no events)**
- 2, **Non Cyclic DFG:** Feedback inputs (Output is fed back to Input), events, etc. are examples for non-acyclic inputs.- A **cyclic graph** contains at least one cycle
- A DFG model translates the program as a single sequential process execution.

Data flow graph construction

original code:

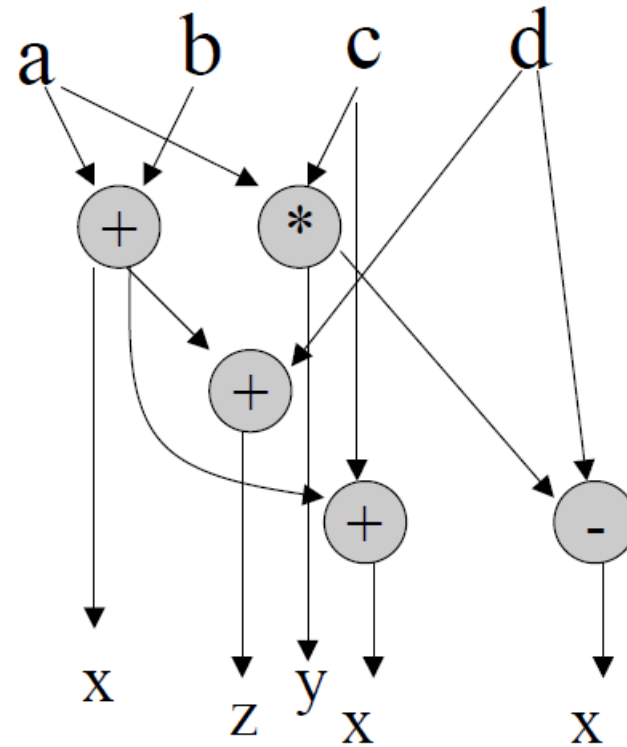
$x \leftarrow a + b;$

$y \leftarrow a * c;$

$z \leftarrow x + d;$

$x \leftarrow y - d;$

$x \leftarrow x + c;$



Data flow graph construction

original code:

$x \leftarrow a + b;$

$y \leftarrow a * c;$

$z \leftarrow x + d;$

$x \leftarrow y - d;$

$x \leftarrow x + c;$

single-assignment form:

$x1 \leftarrow a + b;$

$y \leftarrow a * c;$

$z \leftarrow x1 + d;$

$x2 \leftarrow y - d;$

$x3 \leftarrow x2 + c;$

Data flow graph construction

single-assignment form:

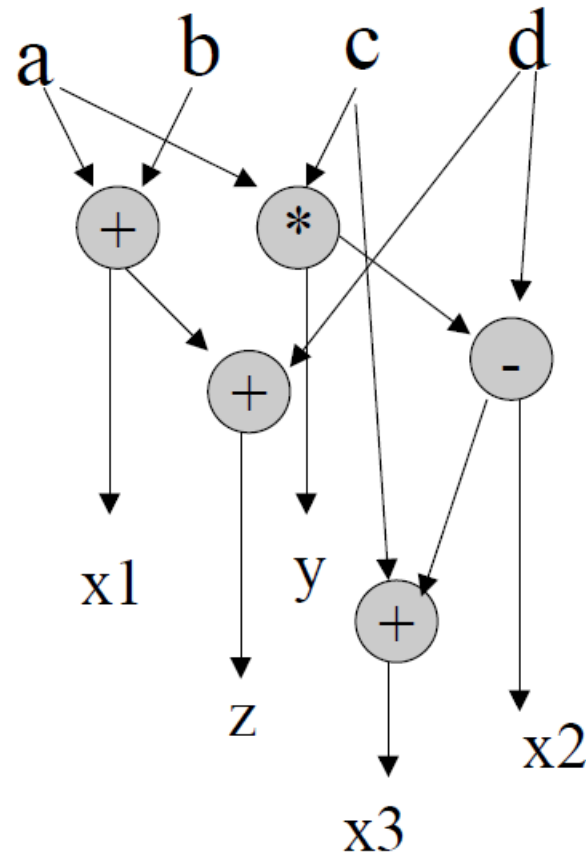
$x1 \leftarrow a + b;$

$y \leftarrow a * c;$

$z \leftarrow x1 + d;$

$x2 \leftarrow y - d;$

$x3 \leftarrow x2 + c;$



Control Data Flow Graph/Diagram (CDFG) Model

- The DFG model is a data driven model in which the execution is controlled by data and it doesn't involve any control operations (conditionals).
- The Control DFG (CDFG) model is used for modelling applications involving conditional program execution.
- CDFG models contains both data operations and control operations.
- The CDFG uses Data Flow Graph (DFG) as element and conditional (constructs) as decision makers.
- CDFG contains both data flow nodes and decision nodes, whereas DFG contains only data flow nodes.

Control/Data Flow Graph

Definition

– A directed graph that represents the **control dependencies** among the functions

branch

fall-through

– $G=(V,E)$

Nodes (V)

– Encapsulated DFG

– Decision

Arces (E)

– flow of the controls

```
x <= a + b;  
if ( x > 100)  
y <= a * c;  
else  
y <= a + c;  
endif
```

What is Conditional Data Flow Graph Model

- o Contains both data and control operations
 - o Dataflow graph is elements and conditionals are decision makers
 - o This mode has both data nodes and decision nodes whereas DFG only has data nodes
 - o Example. Digital Still Camera
-
- Operation --- Circle
 - Data Flow --- Arrow
 - Control Node --- Diamond

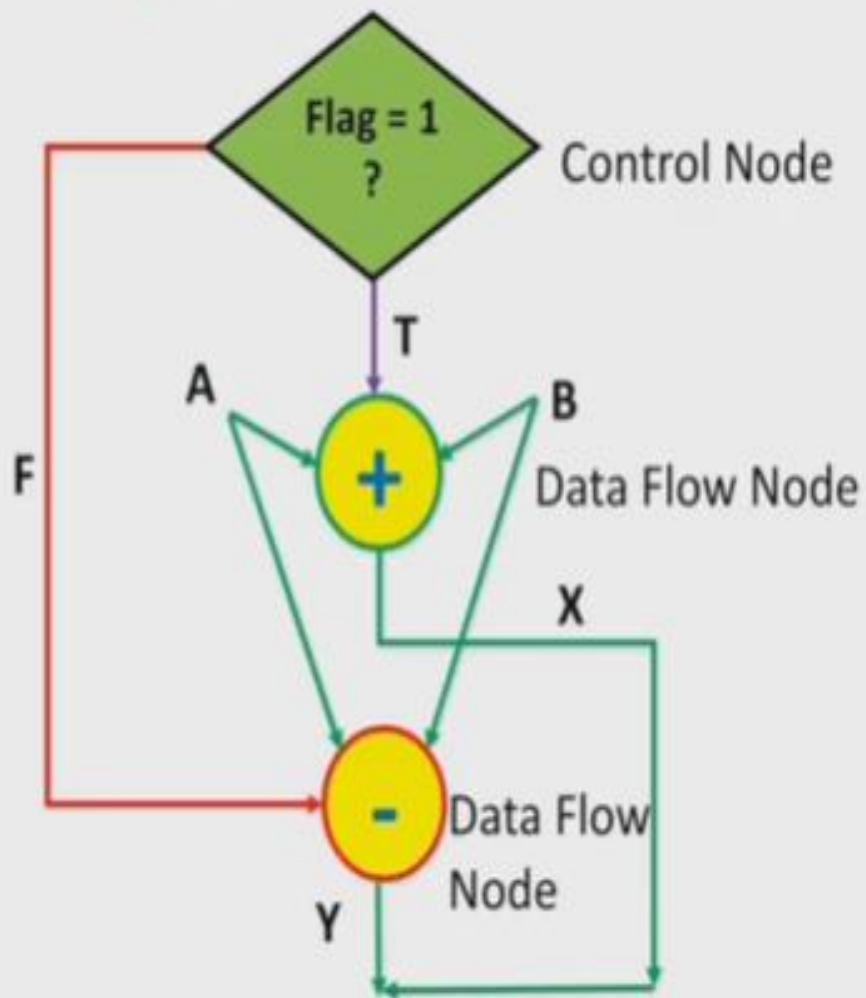
Control Data Flow Graph/Diagram (CDFG) Model (continued)

- Consider the implementation of the CDFG for the following requirement.
- *If $flag = 1, x = a + b$; else $y = a - b$;*
- This requirement contains a decision making process.
- The CDFG model for the same is given in the figure.
- The control node is represented by a 'Diamond' block which is the decision making element in a normal flow chart based design.
- CDFG translates the requirement, which is modelled to a concurrent process model.
- The decision on which process is to be executed is determined by the control node.

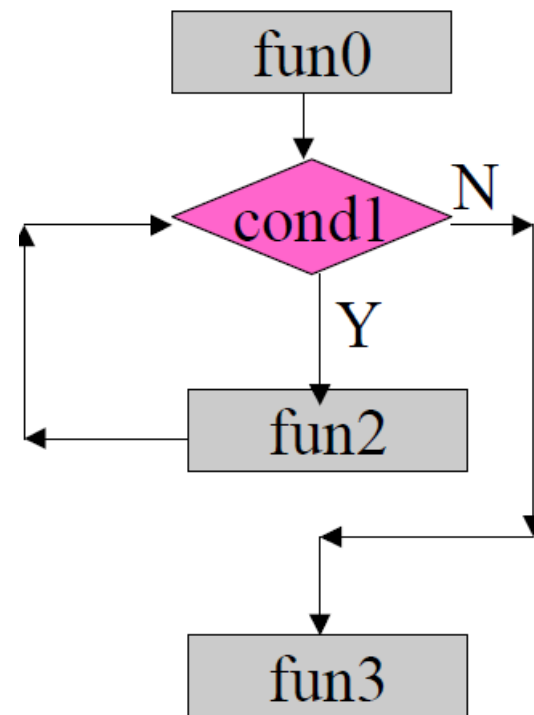
<https://www.geeksforgeeks.org/software-engineering-control-flow-graph-cfg/>

Example of Conditional Data Flow Graph Model

□ If **Flag = 1**, $X = A+B$ else $Y = A-B$

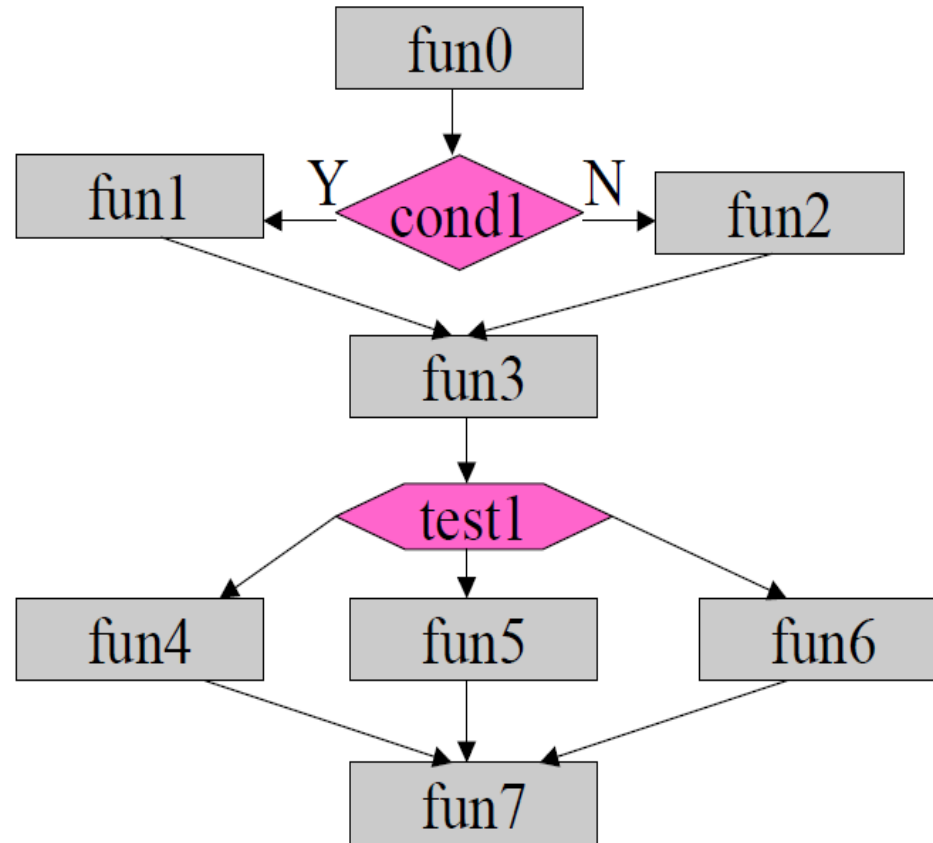


```
fun0();  
while(cond1) {  
    fun1();  
}  
fun2();
```



CDFG Example

```
fun0();  
if (cond1) fun1();  
else fun2();  
fun3();  
switch(test1) {  
  case 1: fun4();  
    break;  
  case 2: fun5();  
    break;  
  case 3: fun6();  
    break;  
}  
fun7();
```

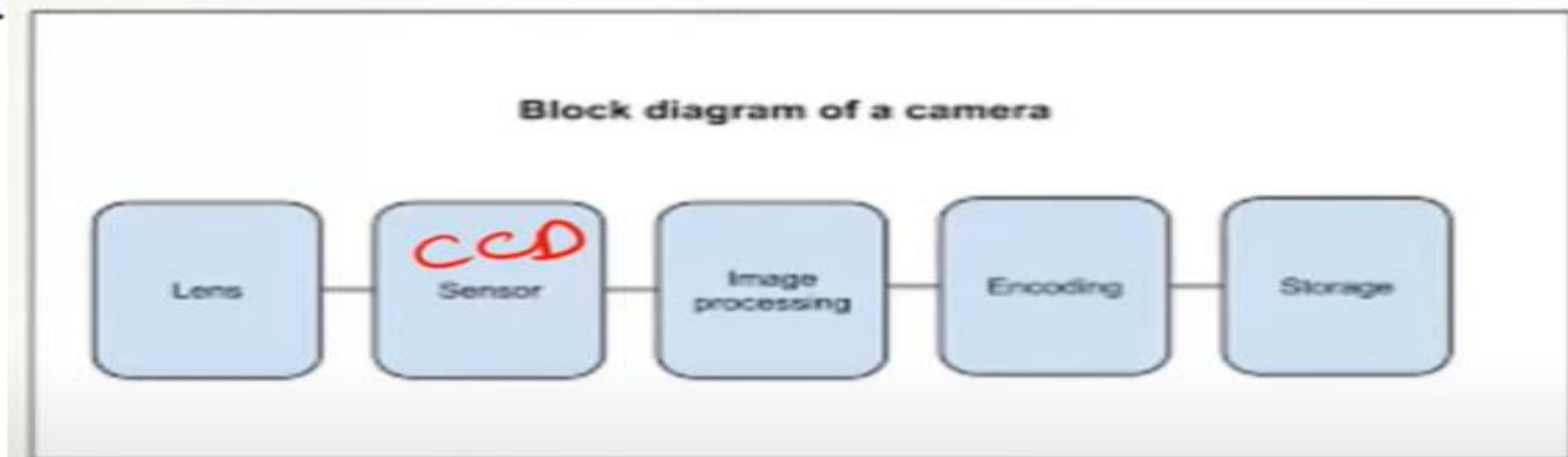


Explain with suitable example the difference between Data Flow Graph and Control Flow Graph

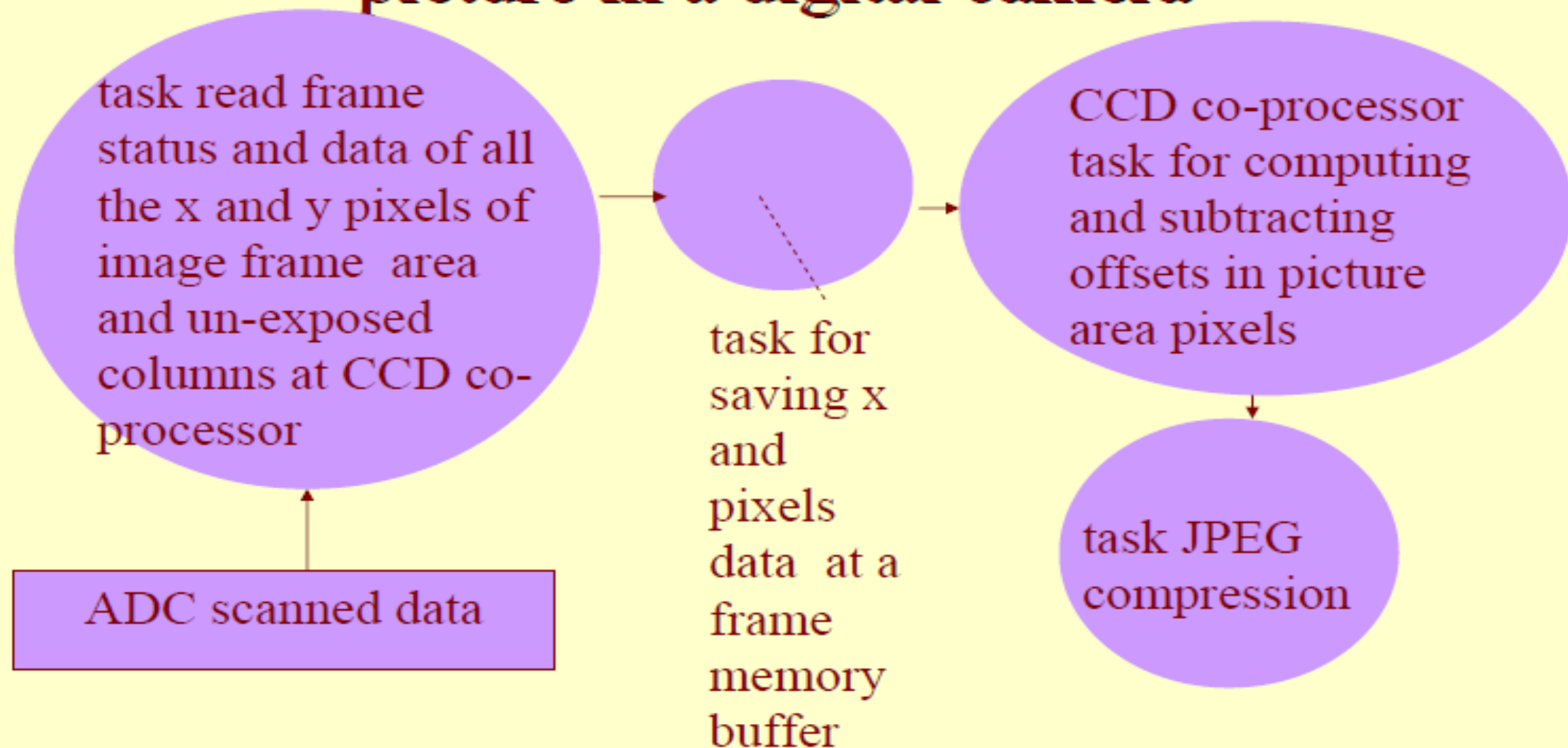
Compute the largest number among three numbers

Control Data Flow Graph/Diagram (CDFG) Model (continued)

- A real world example for modelling the embedded application using CDFG is capturing and saving of the image to a format set by the user in a digital still camera.
- Here everything is data driven.
 - Analog Front End converts the CCD sensor generated analog signal to Digital Signal
 - The data from ADC is stored to a frame buffer for the use of a media processor which performs various operations like, auto correction, white balance adjusting, etc.
- The decision on, in which format the image is stored (formats like JPEG, TIFF, BMP, etc.) is controlled by the camera settings, configured by the user.



DFG model for program for saving a picture in a digital camera



State Machine Model

- The State Machine Model is used for modelling **reactive or event-driven** embedded systems ***whose processing behaviour are dependent on state transitions.***
- Embedded systems used in the control and industrial applications are typical examples for event driven systems.
- **The State Machine model describes the system behaviour with 'States', 'Events', 'Actions' and 'Transitions'.**
 - **State** is a representation of a current situation.
 - An **event** is an input to the state. The event acts as stimuli for state transition.
 - **Transition** is the movement from one state to another.
 - **Action** is an activity to be performed by the state machine.

Finite State Machine (FSM) Model

- • A Finite State Machine (FSM) model is one in which the number of states are finite.
- • The system is described using a finite number of possible states.
- • As an example, let us consider the design of an embedded system for driver/passenger **'Seat Belt Warning'** in an automotive using the FSM model.
- The system requirements are captured as.
- 1. When the vehicle ignition is turned on and the seat belt is not fastened within 10 seconds of ignition ON, the system generates an alarm signal for 5 seconds.
- 2. The Alarm is turned off when the alarm time (5 seconds) expires or if the driver/passenger fastens the belt or if the ignition switch is turned off, whichever happens first.

Finite State Machine (FSM) Model (continued)

Here the **states** are

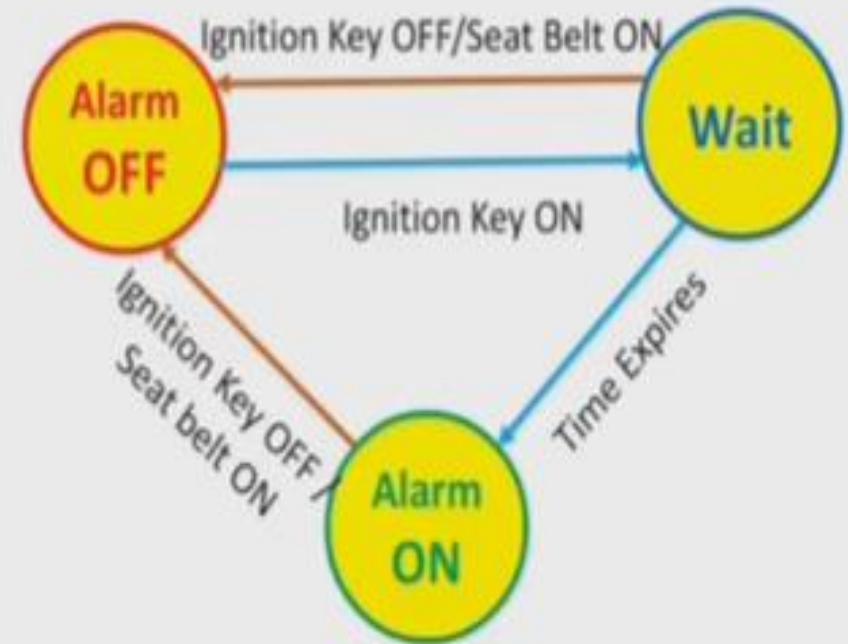
- **'Alarm Off'**
- **'Waiting'**
- **'Alarm On'**

The **events** are

- **'Ignition Key ON'**
- **'Ignition Key OFF'**
- **'Timer Expire'**
- **'Alarm Time Expire'**
- **'Seat Belt ON'**

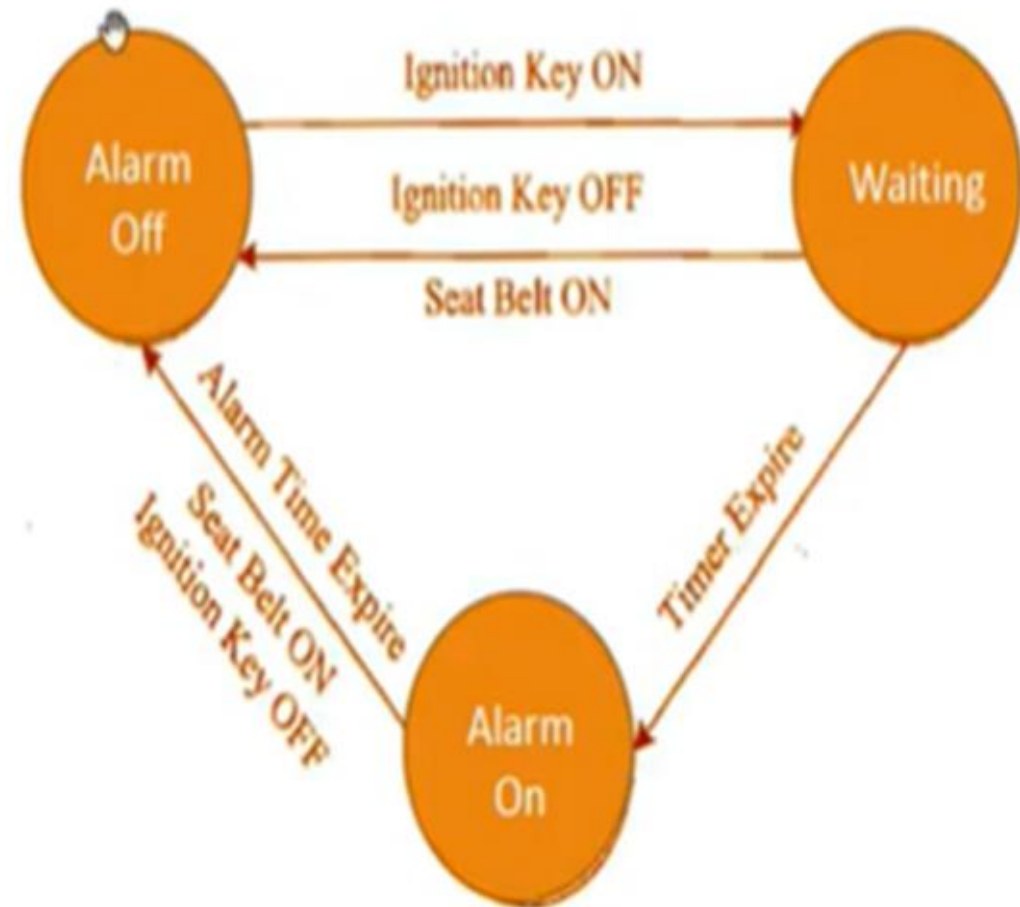
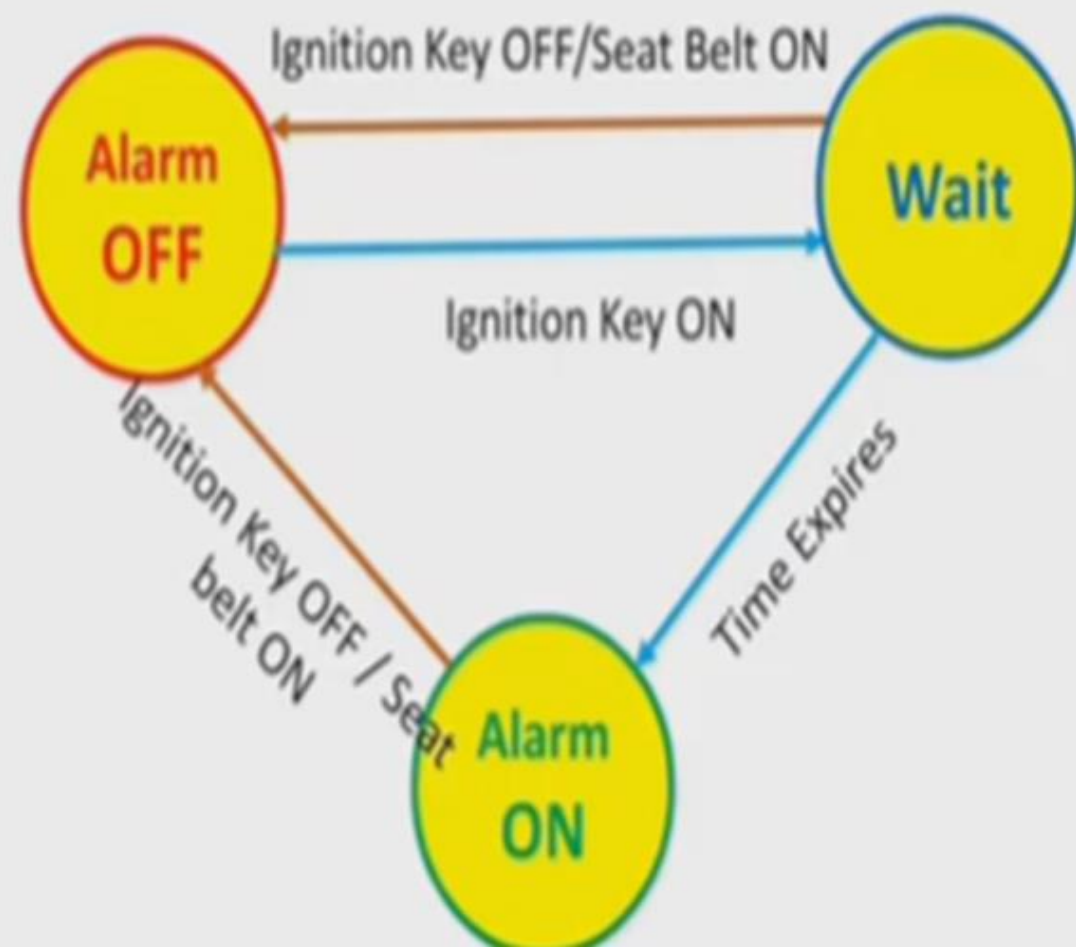
What Is Finite State Machine Model

- ❑ It is one in which number of states are finite
- ❑ Example: Embedded system for driver seat belt warning in automotive



- Using the FSM, the system requirements can be modeled – figure?

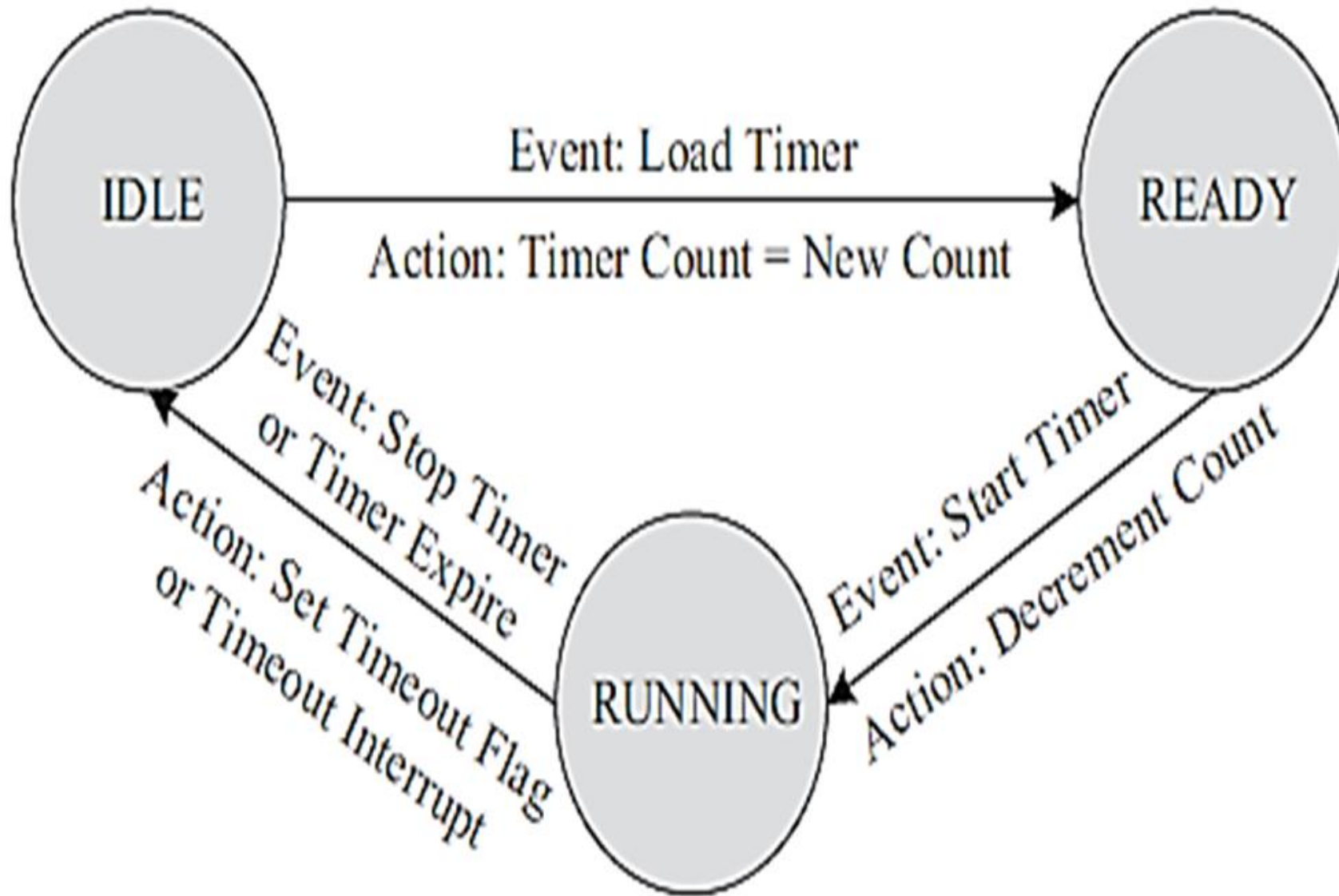
□ Here we can decide the **states** are Alarm Off, Waiting, Alarm ON and **events** are Ignition Key ON, Ignition Key OFF, Waiting Time Expires and Seat Belt ON using FMS



FSM Model for Automatic Seat Belt Warning System

Finite State Machine (FSM) Model (continued)

- The 'Ignition Key ON' event triggers the 10 second timer and transitions the state to 'Waiting'.
- If a 'Seat Belt ON' or 'Ignition Key OFF' event occurs during the wait state, the state transitions into 'Alarm Off'.
- When the wait timer expires in the waiting state, the event 'Timer Expire' is generated and it transitions the state to 'Alarm On' from the 'Waiting' state.
- The 'Alarm On' state continues until a 'Seat Belt ON' or 'Ignition Key OFF' event or 'Alarm Time Expire' event, whichever occurs first.
- The occurrence of any of these events transitions the state to 'Alarm Off'.
- The wait state is implemented using a timer.
 - The timer also has certain set of states and events for state transitions.
 - Using the FSM model, the timer can be modelled as shown in the figure.



FSM Model for timer

'IDLE' state- During the normal condition when the timer is not running it is said to be in the.

'READY' state - when the timer is loaded with the count corresponding to the required time delay. The timer remains in the 'READY' state until a 'Start Timer' event occurs.

'RUNNING' state - on receiving a 'Start Timer' event and remains in the 'RUNNING' state until the timer count expires or a 'Stop Timer' even occurs.

Finite State Machine (FSM) Model (continued)

- As seen from the FSM, the timer state can be either 'IDLE' or 'READY' or 'RUNNING'.
- During the normal condition when the timer is not running, it is said to be in the 'IDLE' state.
- The timer is said to be in the 'READY' state when the timer is loaded with the count corresponding to the required time delay.
- The timer remains in the 'READY' state until a 'Start Timer' event occurs.
- The timer changes its state to 'RUNNING' from the 'READY' state on receiving a 'Start Timer' event and remains in the 'RUNNING' state until the timer count expires or a 'Stop Timer' event occurs.
- The timer state changes to 'IDLE' from 'RUNNING' on receiving a 'Stop Timer' or 'Timer Expire' event.

Sequential Program Model (continued)

- The execution of functions in a sequential program model for the 'Seat Belt Warning' system is illustrated below:

```
#define ON 1
#define OFF 0
#define YES 1
#define NO 0
void seat_belt_warn()
{
    wait_10sec();
    if (check_ignition_key()==ON)
    {
        if (check_seat_belt()==OFF)
        {
            set_timer(5);
            start_alarm();
            while ((check_seat_belt()==OFF)&&(check_ignition_key()==ON)&&(timer_expire()==NO));
            stop_alarm();
        }
    }
}
```

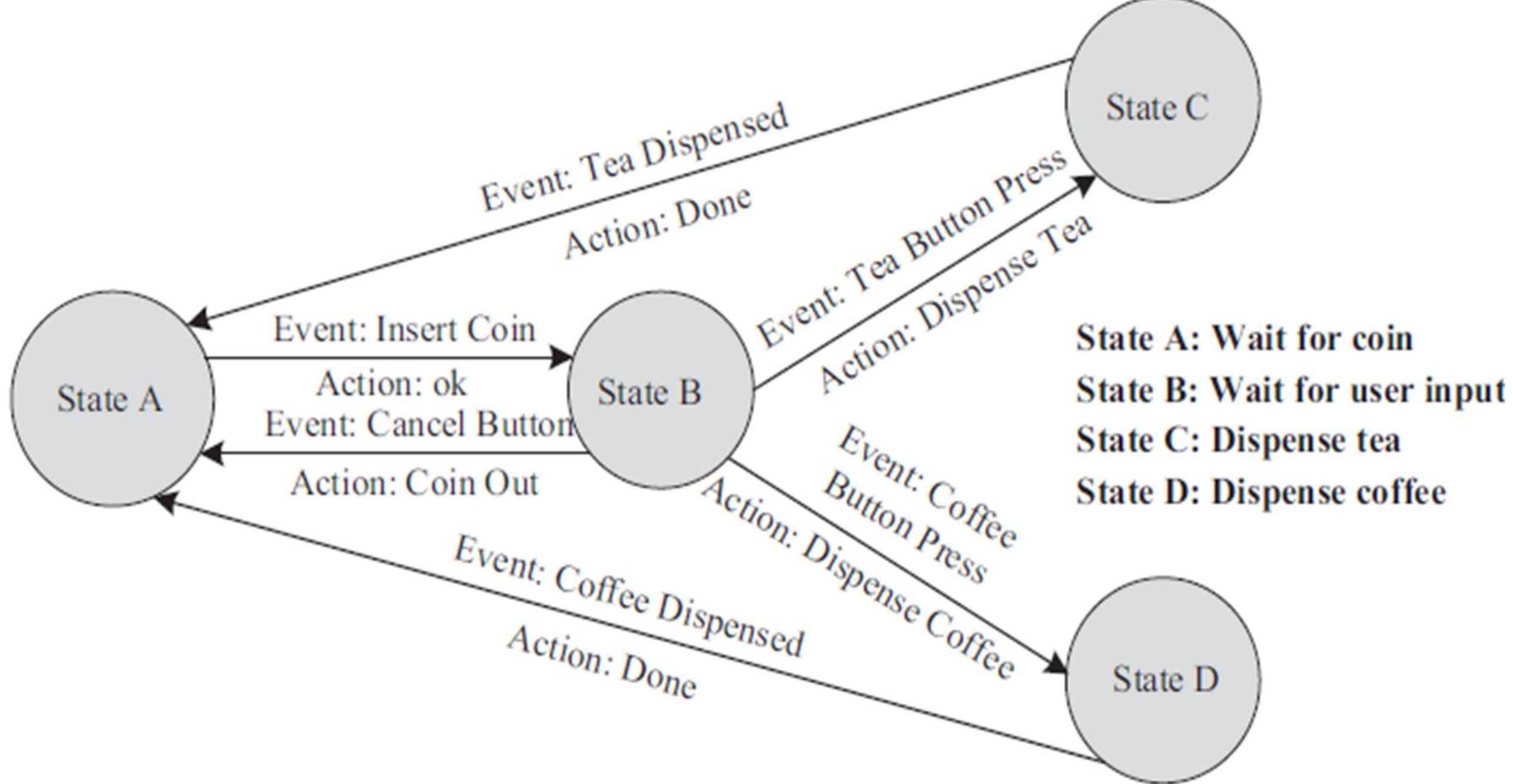
FSM Model - Example 1

Design an automatic tea/coffee vending machine based on FSM model for the following requirement.

- The tea/coffee vending is initiated by user inserting a 5 rupee coin.
- After inserting the coin, the user can either select 'Coffee' or 'Tea' or press 'Cancel' to cancel the order and take back the coin.

Solution

- The FSM Model contains four states namely,
 - 'Wait for coin'
 - 'Wait for User Input'
 - 'Dispense Tea'
 - 'Dispense Coffee'



FSM Model for Automatic Tea\Coffee Vending Machine

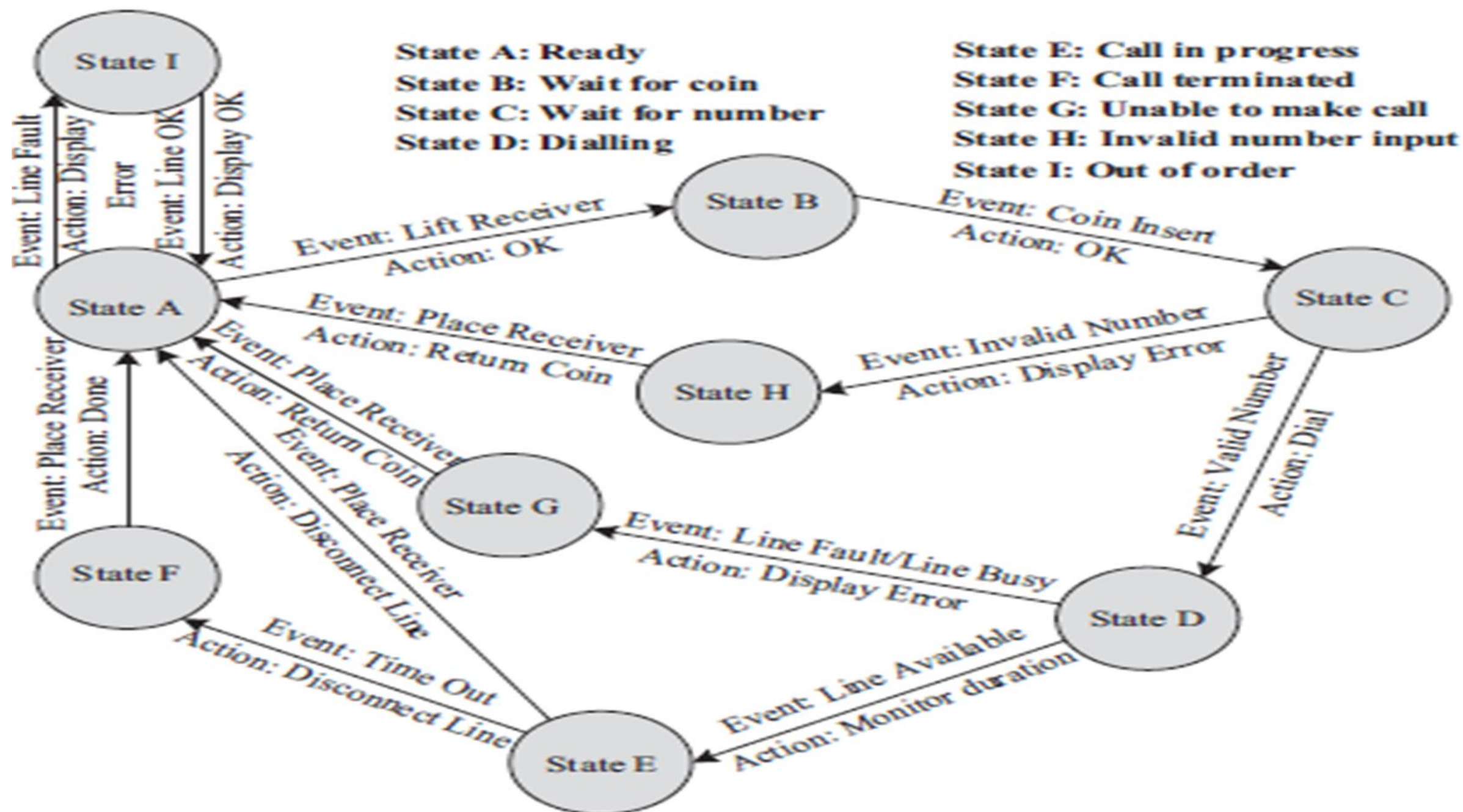
FSM Model - Example 1 (continued)

- The event 'Insert Coin' (5 rupee coin insertion), transitions the state to 'Wait for User Input'.
- The system stays in this state until a user input is received from the buttons 'Cancel', 'Tea' or 'Coffee' (Tea and Coffee are the drink select button).
- If the event triggered in 'Wait State' is 'Cancel' button press, the coin is pushed out and the state transitions to 'Wait for Coin'.
- If the event received in the 'Wait State' is either 'Tea' button press, or 'Coffee' button press, the state changes to 'Dispense Tea' and 'Dispense Coffee' respectively.
- Once the coffee/tea vending is over, the respective states transition back to the 'Wait for Coin' state.

Design a coin operated public telephone unit based on FSM model for the following requirements.

1. The calling process is initiated by lifting the receiver (off-hook) of the telephone unit
2. After lifting the phone the user needs to insert a 1 rupee coin to make the call.
3. If the line is busy, the coin is returned on placing the receiver back on the hook (on-hook)
4. If the line is through, the user is allowed to talk till 60 seconds and at the end of 45th second, prompt for inserting another 1 rupee coin for continuing the call is initiated
5. If the user doesn't insert another 1 rupee coin, the call is terminated on completing the 60 seconds time slot.
6. The system is ready to accept new call request when the receiver is placed back on the hook (on-hook)
7. The system goes to the 'Out of Order' state when there is a line fault

FSM Model for Coin Operated Telephone System



Finite State Machine to Programming for password door open close example

<https://sites.google.com/vt.edu/introduction-to-embeddedsystem/finite-state-machines/basics-fsm>

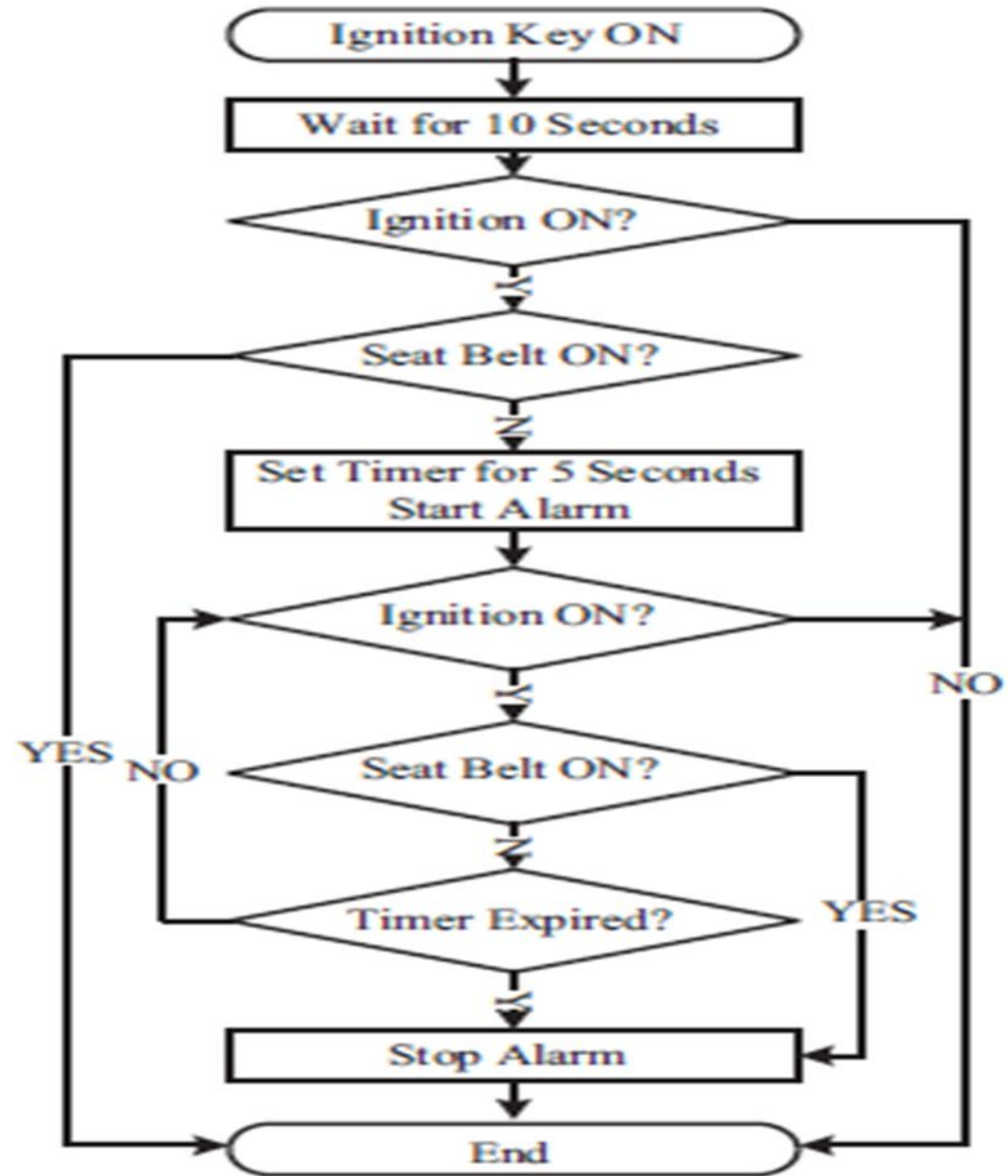
<https://slideplayer.com/slide/17984142/> Elevator

Suppose the passcode is 2534. Thus, the only possible way to unlock the door is by inputting 2, then 5, then 3, then 4 *in that exact order*. Inputting 2 and then another number that is not 5 will reset the lock, forcing you to start over from the beginning. Each time you guess a digit, it is either right or wrong, which means there are only *two* possible transitions from leaving a state. As for our output, the door is either unlocked (U) or locked (!U).

Seat Belt Warning' system in concurrent processing model

1. Timer task for waiting 10 seconds (wait timer task)
2. Task for checking the ignition key status (ignition key status monitoring task)
3. Task for checking the seat belt status (seat belt status monitoring task)
4. Task for starting and stopping the alarm (alarm control task)
5. Alarm timer task for waiting 5 seconds (alarm timer task)

Seat Belt Warning System



7.7 Sequential Program Model for seat belt warning system

- Sequential execution leads to a single sequential execution of task
- leads to poor processor utilization, when the task involves I/O waiting, sleeping for specified duration etc.

concurrent or communicating process mode

- commonly used for the modelling of 'Real Time' systemsconcurrently executing tasks/processes.
- Sequential execution leads to a single sequential execution of task
- Hence, leads to poor processor utilization, when the task involves I/O waiting, sleeping for specified duration etc.

concurrent process mode

- split the task into multiple subtasks
- tackle the CPU usage effectively, when the subtask under execution goes to a wait or sleep mode, by switching the task execution.
- requires additional overheads in task scheduling, task synchronisation and communication.

concurrent processing model

- commonly used for the modelling of 'Real Time' systems.
- Various techniques like 'Shared memory', 'Message Passing', 'Events', etc. are used for communication and synchronising between concurrently executing processes.

Seat Belt Warning' system in concurrent processing model

1. Timer task for waiting 10 seconds (wait timer task)
2. Task for checking the ignition key status (ignition key status monitoring task)
3. Task for checking the seat belt status (seat belt status monitoring task)
4. Task for starting and stopping the alarm (alarm control task)
5. Alarm timer task for waiting 5 seconds (alarm timer task)

Seat Belt Warning' system in concurrent

Tasks for 'Seat Belt Warning System

Create and initialise events

wait_timer_expire, ignition_on, ignition_off,

seat_belt_on, seat_belt_off,

alarm_timer_start, alarm_timer_expire

Create task *Wait Timer*

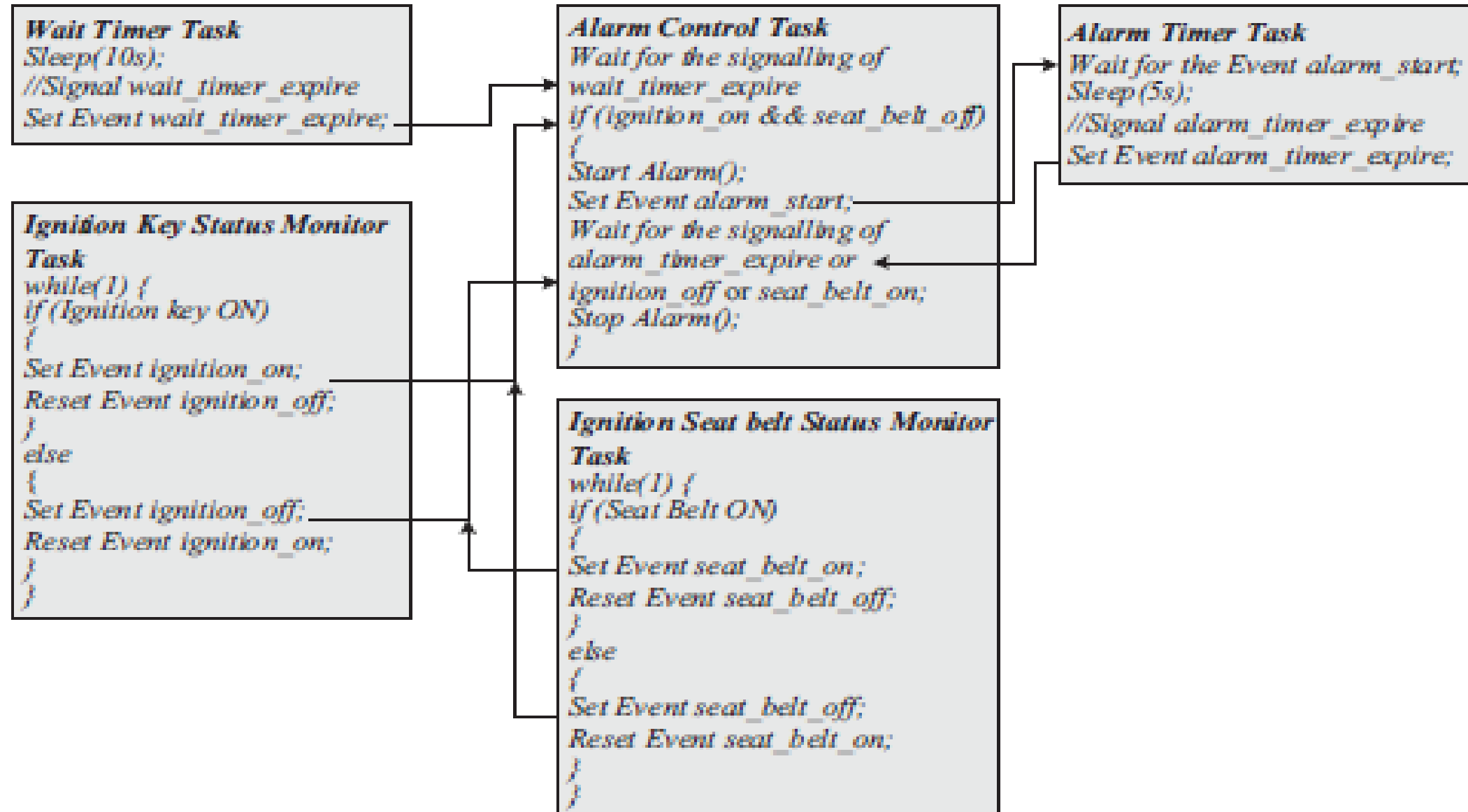
Create task *Ignition Key Status Monitor*

Create task *Seat Belt Status Monitor*

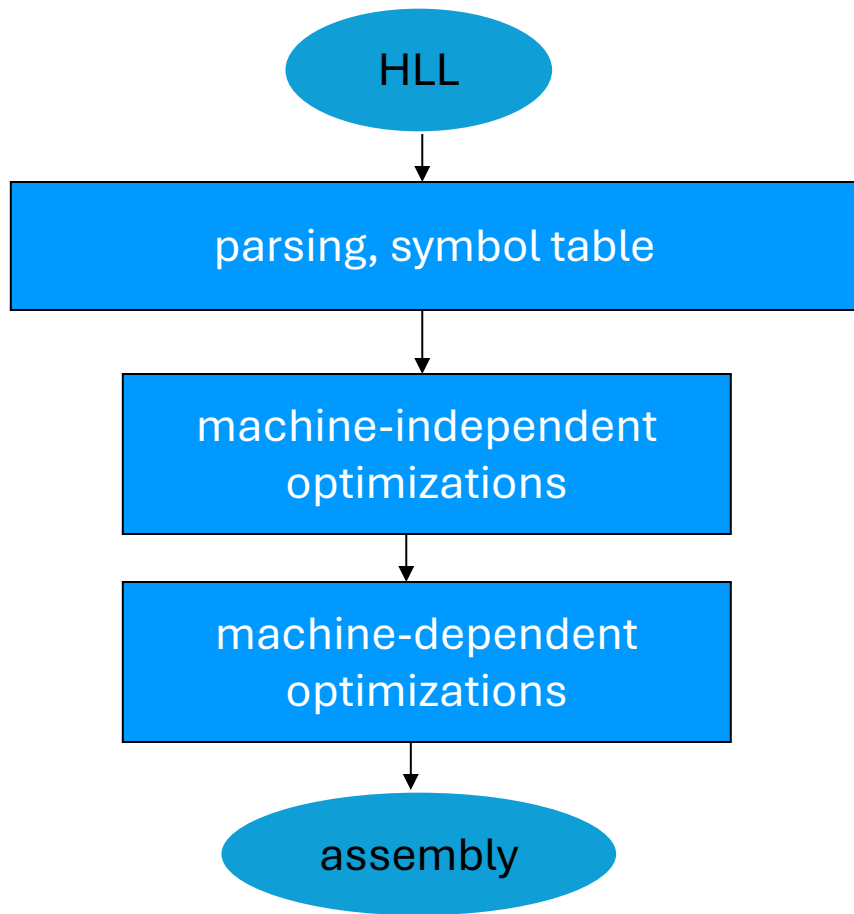
Create task *Alarm Control*

Create task *Alarm Timer*

Concurrent processing Program model for



Basic compilation phases



Statement translation and optimization

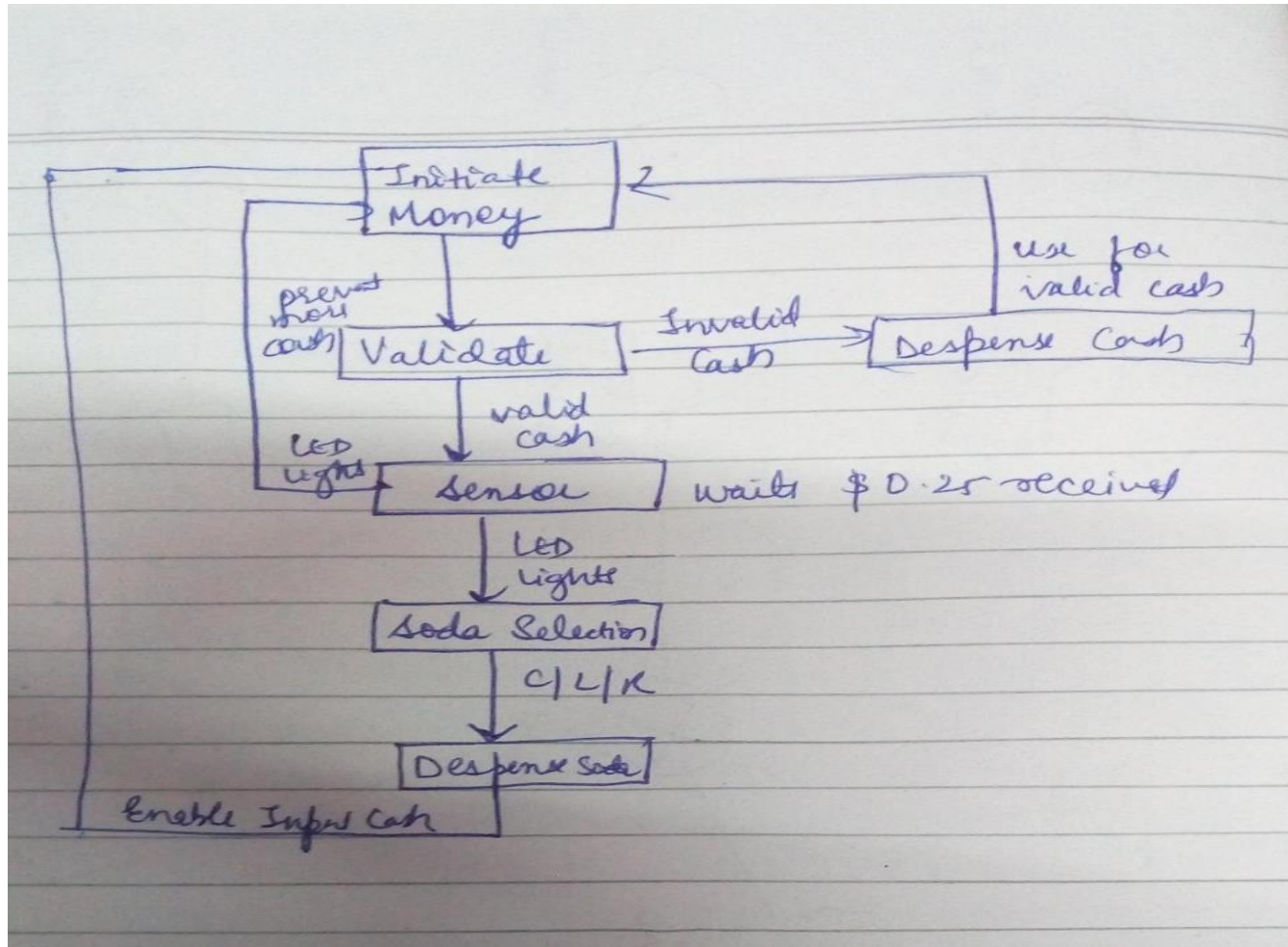
- Source code is translated into intermediate form such as CDFG.
- CDFG is transformed/optimized.
- CDFG is translated into instructions with optimization decisions.
- Instructions are further optimized.

Program design and analysis

- Program-level performance analysis.
- Optimizing for:
 - Execution time.
 - Energy/power.
 - Program size.
- Program validation and testing.
- Safety and security.

A vending machine has three different types of soda: Cola, Lemon-Lime, and Root Beer to choose. Each of the sodas costs \$0.25, and the machine only accepts Quarters, Dimes, and Nickels. When money (a quarter or dime or nickel) is put in, the signal that represents the money is received will stay 1 for only one clock cycle. When at least \$0.25 has been received, the machine lights a LED to indicate a soda flavor may be selected and prevents insertion of more money. The choices by push-buttons are C (Cola), L (Lemon-Lime), and R (Root Beer). When one push-button is pushed, the selected soda is dispensed and the machine returns to its initial state.

Flow chart



<https://guides.visual-paradigm.com/visualizing-system-behavior-a-practical-guide-to-state-diagrams-with-examples/>

<https://medium.com/swlh/vending-machine-design-a-state-design-pattern-approach-5b7e1a026cd2>

<https://www.slideshare.net/slideshow/vending-machine-24580255/24580255>

27-5: Draw the state diagram of the circuit described below, with detailed explanations:

You are to design the main logic of a simple automatic vending machine. The soda cans inside the machine cost ¢25, and the machine accepts only nickels, dimes, and quarters. An electromechanical subsystem accepts the coins sequentially. The logic circuit generates an output signal of “1” that delivers a soda can whenever the total amount received is ¢25 or more. After the selection is made, the circuit is reset.

<https://www.aircconline.com/vlsics/V3N2/3212vlsics02.pdf>

<https://thunderwiring.wordpress.com/fsm-vending-machine-in-vhdl/>